

Trabajo Práctico – Lombok, DTO y Frontend TypeScript

Objetivo general

Desarrollar una pequeña aplicación cliente-servidor que permita gestionar pedidos de una tienda online.

El backend estará desarrollado en Java con Lombok y DTOs, y el frontend será una aplicación sencilla en TypeScript, que consuma datos desde el backend a través de una API REST.

Objetivos específicos

- Aplicar Lombok en las entidades Java para reducir código repetitivo.
- Implementar el patrón DTO para desacoplar la capa de presentación de la lógica interna.
- Exponer una API REST usando Servlets o JAX-RS.
- Desarrollar un frontend en TypeScript que realice peticiones HTTP (GET y POST) al backend.
- Visualizar la información transformada por los DTOs en el navegador.

Objetivos Anexos

Integrar acceso a datos y persistencia con JPA

Parte 1: Backend con Lombok y DTO

El alumno debe crear un proyecto Maven llamado tienda-lombok-dto, implementando las clases Producto, Cliente, Pedido, ProductoDTO y PedidoDTO, todas con anotaciones de Lombok.

Además, debe incluir un servlet que exponga los datos en formato JSON mediante una URL accesible desde el frontend.

```
tienda-lombok-dto/
├─ src/main/java/com/tienda/model/
│   ├── Producto.java
│   ├── Cliente.java
│   └── Pedido.java
│
├─ src/main/java/com/tienda/dto/
│   ├── ProductoDTO.java
│   └── PedidoDTO.java
│
├─ src/main/java/com/tienda/mapper/
│   └── MapperUtil.java
│
├─ src/main/java/com/tienda/servlet/
│   └── PedidoServlet.java
│
└─ pom.xml
```

Clases a usar, agregue las anotaciones correspondientes a las mismas:

```
public class Producto {
    private Long id;
    private String nombre;
    private String categoria;
    private Double precio;
}

public class Cliente {
    private Long id;
    private String nombre;
    private String email;
    private String telefono;
    private List<Pedido> pedidos;
}
```

```
public class Pedido {  
    private Long id;  
    private Cliente cliente;  
    private List<Producto> productos;  
    private LocalDate fecha;  
    private Double total;  
}
```

----- **DTO** -----

```
public class ProductoDTO {  
    private String nombre;  
    private Double precio;  
}
```

```
public class PedidoDTO {  
    private Long id;  
    private String clienteNombre;  
    private List<ProductoDTO> productos;  
    private String fechaPedido;  
    private Double total;  
}
```

NOTA: Agregue las clases adicionales que considere para poder realizar el acceso a los datos de una tabla de base de datos y las clases necesarias para retornar los datos via api rest, estructurando los datos como un JSON.

Métodos estáticos de Conversión de MapperUtil

```
public static ProductoDTO toProductoDTO(Producto producto) {  
    return ProductoDTO.builder()  
        .nombre(producto.getNombre())  
        .precio(producto.getPrecio())  
        .build();  
}  
  
public static PedidoDTO toPedidoDTO(Pedido pedido) {  
    return PedidoDTO.builder()  
        .clienteNombre(pedido.getCliente().getNombre())  
        .productos(pedido.getProductos().stream()  
            .map(MapperUtil::toProductoDTO)  
            .collect(Collectors.toList()))  
        .total(pedido.getTotal())  
        .fechaPedido(FuncionApp.getFechaString(pedido.fecha))  
        .build();  
}
```

FuncionApp.getFechaString es un método que deberá codificar y convierte una LocalDate a un String

Parte 2: Frontend en TypeScript

El alumno desarrollará un frontend simple compuesto por archivos HTML, CSS y TypeScript.

Al presionar un botón en la interfaz, se deberá realizar una solicitud HTTP al backend y mostrar una lista de pedidos mostrando 3 columnas fecha del pedido, nombre del cliente y total del pedido.

Agregue para cada fila una columna con un icono “Ver Detalle ” que al hacer click sobre el mismo ejecute una función que envíe como parámetro el id del pedido y retorne el detalle de los productos involucrados en el mismo y muéstrellos por pantalla.

Estructura esperada del proyecto

Backend (Java Maven):

- com.tienda.model (Producto, Cliente, Pedido)
- com.tienda.dto (ProductoDTO, PedidoDTO)
- com.tienda.servlet (PedidoServlet)

Frontend (TypeScript):

- index.html
- styles.css
- app.ts
- tsconfig.json

Entrega esperada

El alumno debe entregar:

1. Proyecto Java (backend) funcional con Lombok, DTO, acceso a datos y rest.
2. Carpeta frontend con archivos TypeScript y HTML funcionando.